Akademia Nauk Stosowanych w Nowym Sączu
Wydział Nauk Inżynieryjnych • Informatyka • Studia I stopnia

Niezawodność systemów informatycznych

Zajęcia projektowe nr 4, blok P3

Testy funkcjonalne

1 Informacje organizacyjne

- **Blok tematyczny z karty zajęć:** P3: Testy funkcjonalne i niefunkcjonalne
- **Punkt wyjścia:** dokumentacja SUT (`sut_dokumentacja.pdf`), nie kod źródłowy

Cel dzisiejszych zajęć

Po dzisiejszych zajęciach każdy zespół powinien:

1. rozumieć różnicę między testami czarnoskrzynkowymi a testami strukturalnymi z poprzednich zajęć,
2. umieć wyznaczyć klasy równoważności i wartości brzegowe dla wybranych parametrów metod SUT,
3. umieć zbudować tablicę decyzyjną dla logiki zależnej od kilku warunków jednocześnie,
4. mieć zestaw testów funkcjonalnych pokrywających wyznaczone klasy i przypadki brzegowe.

Zmiana perspektywy względem zajęć nr 2 i 3

Testy strukturalne patrzyły na kod i pytały, czy każda jego linia i gałąź zostały wykonane. Testy funkcjonalne odwracają pytanie: patrzą na specyfikację (co metoda ma robić) i pytają, czy wszystkie sensowne kategorie danych wejściowych zostały sprawdzone, niezależnie od tego, jak wygląda implementacja. Dlatego punktem wyjścia jest dziś dokumentacja, nie kod.

2 Wprowadzenie teoretyczne

Testy czarnoskrzynkowe

Testy projektowane wyłącznie na podstawie specyfikacji działania systemu (sygnatura metody, opis parametrów, zgłaszane wyjątki), bez zaglądania do implementacji. Osoba piszącą test traktuje system jak czarną skrzynkę: zna wejście i oczekiwane wyjście, nie zna wnętrza.

Klasy równoważności (equivalence partitioning)

Podział możliwych wartości parametru na grupy (klasy), w obrębie których system powinien zachowywać się tak samo. Zamiast testować każdą możliwą wartość, wystarczy przetestować po jednym reprezentancie z każdej klasy, bo zakłada się, że wynik dla innych wartości z tej samej klasy będzie analogiczny.

Przykład klas równoważności

Dla parametru `kwota` w metodzie `wyplac` sensowne klasy to: wartość ujemna, wartość zero, wartość dodatnia mniejsza od salda, wartość dodatnia równa saldu, wartość dodatnia większa od salda, wartość przekraczająca dzienny limit wypłat. Sześć klas, sześć reprezentatywnych testów, zamiast prób z przypadkowymi liczbami.

Analiza wartości brzegowych (boundary value analysis)

Technika uzupełniająca klasy równoważności: zamiast testować dowolną wartość ze środka klasy, testuje się wartości leżące dokładnie na granicy między klasami (np. kwota równa dokładnie saldu, kwota równa dokładnie limitowi dziennemu) oraz tuż obok granicy.

Dlaczego błędy lubią granice

Błędy typu off-by-one (pomylenie `>` z `>=`) ujawniają się właśnie na granicy, nie w środku zakresu. Test z kwotą równą dokładnie saldu konta wykryje taki błąd, test z kwotą wyraźnie mniejszą od salda, nawet wykonany wielokrotnie, nigdy go nie ujawni.

Tablica decyzyjna

Tabela, w której wierszami są kombinacje warunków logicznych wpływających na zachowanie systemu, a kolumnami, jaki wynik powinien dać system dla każdej kombinacji. Przydatna, gdy zachowanie zależy od kilku niezależnych warunków jednocześnie, bo porządkuje wszystkie kombinacje i zapobiega pominięciu którejś z nich.

Szkic tablicy decyzyjnej dla wypłaty

Konto zablokowane	Kwota dodatnia	Wystarczające środki	W granicach limitu	Oczekiwany wynik
tak	–	–	–	`KontoZablokowane`
nie	nie	–	–	`ValueError`
nie	tak	nie	–	`ValueError`
nie	tak	tak	nie	`ValueError`
nie	tak	tak	tak	wypłata wykonana

Znak „–” oznacza, że dany warunek nie ma znaczenia, bo wcześniejszy warunek już przesądza wynik.

3 Zadanie projektowe na dziś

1. **Wybierzcie dwie lub trzy metody** swojego wariantu SUT (inne niż te szczegółowo analizowane na zajęciach nr 2 i 3, albo te same, ale z nowej perspektywy).
2. **Wyznaczcie klasy równoważności** dla parametrów wejściowych tych metod, na podstawie dokumentacji, nie kodu.
3. **Wyznaczcie wartości brzegowe** dla każdej klasy, w szczególności tam, gdzie dokumentacja wspomina o limitach albo warunkach brzegowych.
4. **Zbudujcie tablicę decyzyjną** dla co najmniej jednej metody, której zachowanie zależy od kilku warunków jednocześnie (np. `wyplac` albo `przelew`).
5. **Napiszcie testy** pokrywające wszystkie wyznaczone klasy, wartości brzegowe i wiersze tablicy decyzyjnej.

6. **Udokumentujcie**, czy któryś test ujawnił zachowanie niezgodne z dokumentacją.

Częsty błąd

Mylenie testów czarnoskrzynkowych z testami strukturalnymi: dobieranie danych wejściowych na podstawie podejrzenia, co widać w kodzie, zamiast na podstawie samej specyfikacji. To zaciera różnicę między dzisiejszymi zajęciami a zajęciami nr 2 i 3, oraz pomija przypadki, które specyfikacja wymaga, a które akurat nie rzucają się w oczy w kodzie.

Co trafia do raportu

Na koniec dzisiejszych zajęć zespół dopisuje do wspólnego raportu projektowego:

- listę wyznaczonych klas równoważności i wartości brzegowych dla przetestowanych metod,
- tablicę decyzyjną wraz z testami ją pokrywającymi,
- listę znalezionych (jeśli są) rozbieżności między dokumentacją a faktycznym zachowaniem,
- wnioski: czy podejście czarnoskrzynkowe ujawniło coś, czego nie wykryły testy strukturalne.

4 Pytania kontrolne

1. Na czym polega podział na klasy równoważności i dlaczego pozwala ograniczyć liczbę testów bez utraty skuteczności?
2. Dlaczego wartości brzegowe statystycznie częściej ujawniają usterki niż wartości ze środka klasy?
3. Jak zbudować tablicę decyzyjną dla funkcji zależnej od trzech niezależnych warunków logicznych? Ile wierszy ma taka tablica w najprostszym przypadku?
4. Czym testy czarnoskrzynkowe różnią się od testów strukturalnych pod względem źródła, na podstawie którego są projektowane?

5 Przygotowanie do zajęć nr 5 (blok P3, ciąg dalszy)

Zajęcia nr 5 kontynuują blok P3 i dotyczą testów niefunkcjonalnych, w tym testów obciążeniowych i współbieżności. Na następne zajęcia:

- zachować dzisiejsze testy funkcjonalne w repozytorium,
- przypomnieć sobie, które metody SUT modyfikują współdzielony stan (saldo konta), to od nich zaczniemy na zajęciach nr 5.